

Script de soutenance

19 slides · répartition, minutage et transitions

Romain · Yannis · Lisa

github.com/RomainJazzar/travelling-merchant

Script de soutenance — Romain, Yannis & Lisa

Le marchand ambulant · TSP · Christofides vs Algorithme génétique 19 slides · durée du script intégral ≈ **21 min 40** hors questions

Répartition et minutage

Intervenant	Slides	Temps	Rôle
Romain	1 – 5, 17, 18	6 min 55	Cadrer le problème et la modélisation, puis assumer les limites et présenter la méthode de travail
Yannis	6 – 10, 16	7 min 25	Christofides de bout en bout, puis la comparaison des deux méthodes
Lisa	11 – 15, 19	7 min 20	L'algorithme génétique de bout en bout, puis la conclusion

Les temps sont calculés sur le nombre de mots réellement écrits, à 150 mots/minute — le débit courant à l'oral en français. Ils sont donc cohérents avec les notes du PowerPoint, qui utilisent le même calcul.

Quatre passages de relais, à soigner :

Relais	Slides	Phrase de bascule
Romain → Yannis	5 → 6	« Yannis va vous détailler la première : Christofides. »
Yannis → Lisa	10 → 11	« Lisa va vous montrer une approche radicalement différente. »
Lisa → Yannis	15 → 16	« Yannis va mettre les deux méthodes face à face. »
Yannis → Romain	16 → 17	« Romain va présenter les limites de notre étude. »
Romain → Lisa	18 → 19	« Lisa va conclure avec notre recommandation. »

(Romain enchaîne lui-même de la slide 17 à la slide 18 : ce n'est pas un relais.)

Conseil de mise en scène. Celui qui prend la parole avance lui-même la slide. C'est le signal le plus clair pour le jury et ça évite les temps morts.

Les chiffres, en un coup d'œil

Dans les scripts ci-dessous les nombres sont **écrits en toutes lettres**, parce que c'est ainsi qu'on les prononce. Voici les mêmes en chiffres, pour vérification :

Christofides	3 445,60 km
Algorithme génétique	3 157,13 km
Écart	288,48 km — 8,37 %

MST (borne inférieure)	2 665,05 km
Couplage	1 142,03 km
Sommets impairs	12
Arêtes du graphe	190

Règles communes

1. **Ne lisez jamais la slide.** Elle porte l'idée, vous portez l'explication.
2. **Un chiffre à la fois.** Laissez une seconde après chaque nombre important.
3. **Dites « observé », pas « optimal ».** Systématiquement.
4. **Regardez le jury,** pas l'écran. Vous connaissez vos slides.
5. **Si vous perdez le fil,** revenez au chiffre de la slide : il relance toujours.

ROMAIN — Slides 1 à 5

SLIDE 1 — Titre

Durée : 40 s

Bonjour. Nous sommes Romain, Yannis et Lisa, et nous allons vous présenter notre projet sur le marchand ambulant — autrement dit, le problème du voyageur de commerce. Le scénario est simple : Théobald est un marchand qui doit visiter vingt villes françaises, une seule fois chacune, puis rentrer à son point de départ. Il veut parcourir le moins de kilomètres possible. Le sujet nous demande d'implémenter deux méthodes très différentes — l'algorithme de Christofides et un algorithme génétique — et de les comparer. À la fin, nous vous dirons laquelle nous recommandons à Théobald, et surtout pourquoi. Je vais commencer par le problème et la modélisation, Yannis prendra Christofides, et Lisa la partie génétique.

Transition. « Commençons par ce qui rend ce problème difficile. »

Question probable après cette slide : aucune, on enchaîne.

SLIDE 2 — Le problème

Durée : 1 min

Le TSP consiste à trouver le cycle le plus court qui passe une fois par chaque ville et revient au départ. Le premier réflexe, c'est de dire : essayons tous les ordres possibles et gardons le meilleur. Le problème, c'est le nombre d'ordres possibles. Si on fixe la ville de départ — parce que dans un cycle, le point de départ n'a aucune importance — il reste **dix-neuf factorielle** tournées. C'est-à-dire environ **cent vingt millions de milliards**. (laisser une seconde) Pour vous donner une idée : même avec une machine capable d'évaluer **un milliard** de tournées par seconde, il faudrait près de **quatre ans** de calcul. Pour une seule instance de vingt villes. Et attention à l'échelle du graphique : elle est **logarithmique**. Chaque graduation vaut dix fois la précédente, donc la vraie explosion est bien plus brutale que ce qu'on voit. C'est ce qu'on appelle un problème **NP-difficile** : on ne connaît aucun algorithme exact qui reste rapide quand le nombre de villes augmente. Notre objectif n'est donc pas de trouver la solution parfaite, mais une très bonne solution en un temps raisonnable.

Transition. « Avant de chercher une solution, il faut traduire ce problème en objet mathématique. »

Question probable : « Pourquoi $(n-1)!$ et pas $n!$? » **Réponse :** « Parce que dans un cycle, le point de départ n'a pas d'importance : Paris-Lyon-Nice-Paris et Lyon-Nice-Paris-Lyon sont le même circuit. Il y a n façons d'écrire le même cycle, donc on divise $n!$ par n . Et si on tient compte en plus du sens de parcours, on descend même à dix-neuf factorielle divisé par deux. »

SLIDE 3 — Modélisation

Durée : 55 s

Pour résoudre le problème, on le traduit en théorie des graphes. Chaque ville devient un **sommet**, chaque liaison possible une **arête**, et le poids de l'arête est la distance entre les deux villes. Notre graphe a trois propriétés importantes. Il est **complet** : toutes les paires de villes sont reliées, parce qu'on sait calculer une distance entre n'importe lesquelles. Théobald n'est bloqué par aucune route manquante. Il est **pondéré** : sans poids sur les arêtes, « le chemin le plus court » n'aurait aucun sens. Et il est **non orienté** : la distance de Lyon à Nice est la même que de Nice à Lyon. Conséquence directe : avec vingt sommets, on a **cent quatre-vingt-dix arêtes** — vingt fois dix-neuf, divisé par deux. On divise par deux précisément parce qu'une arête relie une **paire** de villes, pas un couple ordonné.

Transition. « Reste à savoir comment on calcule concrètement ces cent quatre-vingt-dix distances. »

Question probable : « Pourquoi 190 et pas 380 ? » **Réponse :** « 380 compte les couples ordonnés, c'est-à-dire l'aller et le retour séparément. Comme le graphe est non orienté, l'arête Paris-Lyon et l'arête Lyon-Paris sont la même : on divise par deux. »

SLIDE 4 — Haversine

Durée : 1 min 20

Nos données sont des latitudes et des longitudes : des points sur une **sphère**, pas des coordonnées dans un plan. Si on appliquait Pythagore directement sur ces nombres, on additionnerait des degrés de latitude et des degrés de longitude. Or un degré de latitude vaut toujours à peu près cent onze kilomètres, alors qu'un degré de longitude en vaut cent onze à l'équateur mais seulement **soixante-treize** à la latitude de Paris. On additionnerait donc des unités qui ne valent pas la même chose. La formule de **Haversine** calcule la longueur de l'arc de grand cercle entre deux points : la vraie distance à vol d'oiseau sur la surface de la Terre. On prend un rayon terrestre de six mille trois cent soixante et onze kilomètres. Mais le point vraiment important est à droite. Haversine est une **métrique**. Elle est positive, nulle uniquement entre une ville et elle-même, symétrique, et surtout elle respecte l'**inégalité triangulaire** : passer par une ville intermédiaire ne peut jamais raccourcir le trajet. Retenez bien cette dernière propriété. C'est **exactement** l'hypothèse dont Christofides a besoin pour que sa garantie tienne. Yannis y reviendra. Et c'est aussi pourquoi nous n'avons pas pris de distances routières réelles : les temps de trajet ne respectent pas forcément cette propriété, et on perdrait la garantie.

Transition. « On a un graphe, on a des distances. Il reste à choisir comment chercher. »

Question probable : « Avez-vous vérifié que l'inégalité triangulaire est respectée ? » **Réponse :** « Oui, nous avons testé les six mille huit cent quarante triplets de notre matrice : zéro violation. Ce n'est donc pas une supposition, c'est un fait vérifié. »

SLIDE 5 — Deux stratégies

Durée : 45 s

Il existe deux grandes familles de réponses à ce type de problème, et elles sont radicalement opposées. À gauche, l'approche **constructive** : Christofides. On construit la tournée étape par étape, avec des outils de théorie des graphes. C'est **déterministe**, donc une seule exécution suffit, et surtout on dispose d'une **garantie mathématique prouvée**. À droite, l'approche **exploratoire** : l'algorithme génétique. Là, on ne construit rien du tout. On part de solutions au hasard et on les fait évoluer. C'est **stochastique** : il faut plusieurs exécutions pour parler sérieusement d'un résultat, et il n'y a aucune garantie d'optimalité. Deux philosophies opposées sur exactement la même instance : c'est tout l'intérêt de la comparaison que demande le sujet.

Transition — PASSAGE DE RELAIS.

« Yannis va vous détailler la première : Christofides. »

Question probable : « Pourquoi le sujet impose-t-il ces deux méthodes ? » **Réponse :** « Parce qu'elles illustrent les deux grandes façons d'attaquer un problème NP-difficile : soit on construit avec une garantie, soit on explore sans garantie mais avec plus de liberté. Les comparer, c'est comprendre ce qu'on gagne et ce qu'on perd dans chaque cas. »

YANNIS — Slides 6 à 10

SLIDE 6 — La chaîne Christofides

Durée : 1 min 05

Merci Romain. Voici la chaîne complète de Christofides. Je vous donne d'abord la vue d'ensemble, puis on reprend les étapes clés une par une. On part du graphe complet. On construit un **arbre couvrant minimal**, qui relie toutes les villes au coût le plus faible — mais qui n'est pas une tournée. Dans cet arbre, certains sommets ont un **degré impair**. On les apparie deux à deux au coût le plus bas possible : c'est le **couplage**. En ajoutant ces arêtes à l'arbre, on obtient un **multigraphe où tous les degrés sont pairs**, ce qui garantit l'existence d'un **circuit eulérien**. On le parcourt avec Hierholzer, puis on **supprime les villes déjà visitées**. On obtient enfin un vrai cycle hamiltonien : notre tournée. Ce qu'il faut retenir, c'est que chaque étape corrige un défaut **précis** de la précédente. Et qu'au bout de la chaîne, on a une garantie : la tournée ne dépasse **jamais** une fois et demie l'optimum.

Transition. « Commençons par la première brique : l'arbre couvrant minimal. »

Question probable : « Pourquoi huit étapes pour un seul résultat ? » **Réponse :** « Parce que chacune répare quelque chose. Le MST connecte tout mais n'est pas un circuit. Le couplage rend les degrés pairs. Hierholzer produit un circuit mais qui repasse par des villes. Le raccourci corrige ça. Si on en enlève une seule, la chaîne ne tient plus. »

SLIDE 7 — MST et Prim

Durée : 1 min 20

Un arbre couvrant, c'est un sous-graphe qui touche **toutes** les villes sans jamais former de boucle. Avec vingt sommets, il a exactement **dix-neuf arêtes**. Le minimal, c'est celui dont la somme des poids est la plus faible : chez nous, **deux mille six cent soixante-cinq kilomètres**. Je l'ai implémenté avec l'algorithme de **Prim**. Le principe est glouton : on part d'une ville, et à chaque tour on ajoute l'arête la moins chère qui relie une ville pas encore atteinte. On utilise un tas binaire pour retrouver cette arête rapidement. Attention à ne pas confondre avec **Dijkstra** : Dijkstra minimise la distance depuis un point de départ, Prim minimise le coût total de connexion de tout le réseau. Ce ne sont pas les mêmes arbres. **Kruskal** aurait donné exactement le même poids — c'est simplement une autre façon de le construire. Nous avons d'ailleurs vérifié que le sommet de départ de Prim ne change pas le résultat : nous avons testé les vingt départs possibles, toujours deux mille six cent soixante-cinq kilomètres. Et surtout — c'est le point le plus important de cette slide — ce poids est une **borne inférieure** de l'optimum du TSP. Si on prend la tournée optimale et qu'on lui enlève une arête, on obtient un arbre couvrant, qui coûte donc au moins autant que le minimal. On y reviendra à la fin.

Transition. « Ce squelette n'est pas une tournée. Voyons ce qui lui manque. »

Question probable : « Prim ou Kruskal ? » **Réponse :** « Les deux donnent le même poids. J'ai pris Prim parce que notre graphe est complet, donc dense, et Prim est mieux adapté aux graphes denses. Il ne demande qu'un tas, alors que Kruskal demande en plus une structure union-find. Et le sujet cite Prim dans sa base de connaissances. Mais Kruskal aurait parfaitement convenu. »

SLIDE 8 — Degrés impairs et couplage

Durée : 1 min 15

Le **degré** d'un sommet, c'est son nombre d'arêtes. Dans notre arbre, **douze** villes ont un degré impair — elles sont en doré sur la carte. Pourquoi est-ce un problème ? Parce que pour traverser une ville sans s'y arrêter, il faut y **entrer** par une arête et en **ressortir** par une autre. Les arêtes se consomment par paires. Un sommet de degré impair finit donc toujours par bloquer le parcours. Premier point mathématique : ces sommets sont **forcément en nombre pair**. C'est le lemme des poignées de main. Si on additionne tous les degrés, on compte chaque arête deux fois, donc la somme est paire. Et une somme paire ne peut pas contenir un nombre impair de termes impairs. Ce n'est pas une chance de notre instance, c'est un théorème. Comme ils sont en nombre pair, on peut les **appairer deux à deux**. On cherche le couplage **parfait** — chaque sommet impair dans exactement une paire — et **de poids minimal** — la somme des distances la plus petite possible. Chez nous : six arêtes, **mille cent quarante-deux kilomètres**, en rouge sur la carte. Et voilà pourquoi ça marche : ajouter une arête à un sommet augmente son degré de un. Impair plus un égale pair. Après l'union, **tous** les degrés sont pairs.

Transition. « Et un graphe connexe dont tous les degrés sont pairs a une propriété remarquable. »

Question probable : « Pourquoi le couplage doit-il être de poids minimal ? » **Réponse :** « Deux raisons. La pratique : ces arêtes s'ajoutent au coût final, donc on veut les moins chères. Et la théorique, qui est la vraie : c'est la minimalité qui permet de prouver que le couplage coûte au plus la moitié de l'optimum. Sans elle, on n'aurait pas le facteur un virgule cinq. »

SLIDE 9 — Euler, Hamilton, raccourci

Durée : 1 min 20

Un graphe connexe dont tous les degrés sont pairs possède **toujours** un circuit eulérien : un parcours qui emprunte chaque **arête** exactement une fois et revient au départ. C'est l'algorithme de **Hierholzer** qui le construit. Mais attention, ce n'est pas encore une tournée pour Théobald. Parce qu'un circuit eulérien raisonne en **arêtes**, pas en **sommets** : certaines villes y apparaissent plusieurs fois. Le TSP, lui, cherche un **cycle hamiltonien**, qui passe une seule fois par chaque sommet. C'est la distinction essentielle : **Euler compte les arêtes, Hamilton compte les sommets**. On fait donc la conversion en parcourant le circuit et en **sautant** toute ville déjà visitée. Et c'est là que l'inégalité triangulaire dont parlait Romain revient : si on passait par A, puis B, puis C, et qu'on saute B, on remplace la distance A-B plus B-C par la distance A-C, qui est forcément **inférieure ou égale**. Le raccourci ne rallonge donc jamais la tournée. Chez nous, c'est très concret : le circuit eulérien pèse trois mille huit cent sept kilomètres, et après raccourcis la tournée ne fait plus que trois mille quatre cent quarante-cinq. Les raccourcis ont fait gagner **trois cent soixante et un kilomètres**. Et si on met tout bout à bout : le MST coûte au plus l'optimum, le couplage au plus la moitié de l'optimum, et les raccourcis n'ajoutent rien. D'où la garantie d'**une fois et demie l'optimum** — valable uniquement parce que nos distances sont métriques.

Transition. « Voyons ce que ça donne concrètement sur la carte de Théobald. »

Question probable : « D'où vient exactement le facteur un virgule cinq ? » **Réponse :** « De deux bornes. Un : le MST coûte au plus l'optimum, parce qu'en retirant une arête à la tournée optimale on obtient un arbre couvrant. Deux : le couplage minimal coûte au plus la moitié de l'optimum — on le voit en parcourant la tournée optimale et en ne retenant que les sommets impairs, ce qui donne un cycle de coût au plus OPT, qu'on décompose en deux couplages parfaits dont le moins cher coûte au plus OPT sur deux. La somme donne un virgule cinq, et le raccourci n'ajoute rien. »

SLIDE 10 — Résultat Christofides

Durée : 1 min 05

Voici la tournée que Christofides propose à Théobald. Elle part de Paris, passe par Le Havre, Rennes, Nantes, descend la façade atlantique jusqu'à Toulouse, longe la Méditerranée, remonte par les Alpes et Lyon, puis revient par Dijon, Strasbourg, Reims et Lille. **Trois mille quatre cent quarante-cinq virgule six kilomètres.** Les numéros sur la carte donnent l'ordre de passage : vous pouvez vérifier qu'aucune ville n'apparaît deux fois et qu'on revient bien au point de départ. À droite, le détail du calcul. L'arbre couvrant pesait deux mille six cent soixante-cinq kilomètres. Le couplage a ajouté mille cent quarante-deux, ce qui ferait trois mille huit cent sept. Et les raccourcis ont fait redescendre le total à trois mille quatre cent quarante-cinq. Côté temps : environ une milliseconde et demie sur nos machines. Je précise que c'est une **mesure expérimentale** : sur une autre machine le chiffre changera. Ce qui reste vrai, c'est l'ordre de grandeur — quelques millisecondes.

Transition — PASSAGE DE RELAIS.

« Lisa va maintenant vous montrer une approche radicalement différente. »

Question probable : « Cette tournée est-elle optimale ? » **Réponse :** « Non. Christofides est un algorithme d'**approximation** : il garantit de ne jamais dépasser une fois et demie l'optimum, mais il ne garantit pas de l'atteindre. Et on va justement voir que le génétique fait mieux. »

LISA — Slides 11 à 15

SLIDE 11 — L'algorithme génétique

Durée : 1 min 15

Merci Yannis. L'algorithme génétique, lui, ne construit rien. Il part de solutions **au hasard** et les fait **évoluer**, exactement comme une population biologique. Le vocabulaire est celui de la génétique. Un **individu**, c'est une tournée complète — le sujet parle joliment de « Théobald d'univers parallèles ». Son **chromosome**, c'est la permutation des villes. Un **gène**, c'est une ville à une position donnée. La **fitness**, c'est la distance totale du cycle, retour au départ compris. Et ici on cherche à la **minimiser** : plus la distance est faible, meilleur est l'individu. Une **génération**, c'est un tour complet de la boucle que vous voyez à gauche : on évalue, on sélectionne, on croise, on mute, et on repart. Nous en faisons **cinq cent vingt**, sur une population de **cent soixante** tournées. Un détail d'implémentation important : nous avons **fixé Paris** comme point d'ancrage, et le chromosome ne contient que les dix-neuf autres villes. La raison, c'est qu'une tournée cyclique reste la même si on la décale. Paris-Lyon-Nice-Paris et Lyon-Nice-Paris-Lyon, c'est le même cycle. Fixer un point supprime ces représentations redondantes sans supprimer aucune solution.

Transition. « Regardons maintenant en détail les opérateurs qui font cette évolution. »

Question probable : « Est-ce que fixer Paris force Théobald à partir de Paris ? » **Réponse :** « Non, ça ne change rien à la tournée. Le cycle est le même quel que soit le point d'entrée. S'il habitait Lyon, il parcourrait exactement le même circuit, simplement en le commençant à Lyon, avec la même distance.

SLIDE 12 — Les opérateurs

Durée : 1 min 30

Voici nos quatre opérateurs. Je commence par le **croisement**, en haut à gauche, parce que c'est le plus subtil. Si on prenait un croisement naïf — la première moitié du parent un, la seconde moitié du parent deux — on obtiendrait une tournée **invalid**e : certaines villes apparaîtraient deux fois, d'autres disparaîtraient. Le **Ordered Crossover** règle ça. On conserve un segment du parent un, en doré, puis on complète les cases vides en parcourant le parent deux dans l'ordre, en ignorant les villes déjà présentes. Le résultat est **toujours** une permutation valide — par construction, il n'y a jamais besoin de réparer l'enfant. Ensuite la **mutation**, en bas à gauche. Nous en utilisons deux : l'**échange**, qui permute deux villes, et l'**inversion**, qui retourne un segment entier. L'inversion est particulièrement intéressante parce qu'elle ne modifie que **deux arêtes** de la tournée, alors que l'échange en modifie quatre. C'est exactement le mouvement de base de l'algorithme deux-opt, connu pour décroiser efficacement les itinéraires. La mutation sert à **explorer**. Sans elle, le croisement ne fait que recombinaison l'existant et la population se fige. Trop de mutation, à l'inverse, et la recherche devient du hasard pur. À droite, la **sélection par tournoi** : on tire cinq individus au hasard et on garde le meilleur comme parent. C'est plus simple et plus stable qu'une roulette, et le paramètre k règle la pression de sélection. Enfin l'**élitisme** : les six meilleures tournées passent telles quelles à la génération suivante. C'est ce qui garantit que la meilleure solution connue n'est jamais perdue.

Transition. « Ces opérateurs ont des réglages. Voyons comment nous les avons choisis. »

Question probable : « Pourquoi OX plutôt qu'un autre croisement ? » **Réponse :** « Parce qu'il garantit une permutation valide sans réparation, et parce qu'il transmet l'**ordre relatif** des villes, qui est exactement l'information pertinente pour une tournée. Il existe mieux — ERX, qui raisonne en arêtes — mais c'est nettement plus lourd à implémenter. »

SLIDE 13 — Paramètres et benchmark

Durée : 1 min 15

Le sujet demande explicitement de tester différentes configurations. Nous en avons testé **trois** — rapide, équilibrée, exploratoire — chacune sur **trois seeds** différents : onze, vingt-deux et trente-trois. Un **seed**, c'est la graine du générateur aléatoire. À seed identique, l'exécution est rigoureusement reproductible. Tester plusieurs seeds est indispensable : avec un algorithme stochastique, une seule exécution ne dit **rien** sur la fiabilité. Et regardez le résultat. Les trois configurations atteignent la **même** meilleure distance : trois mille cent cinquante-sept kilomètres. Sur ce seul critère, elles seraient indiscernables. Mais la rapide et l'exploratoire ont chacune un seed qui **échoue** à trois mille trois cent trente-quatre, ce qui leur donne un écart-type de quatre-vingt-trois kilomètres. La configuration **équilibrée**, elle, a un écart-type **nul** : les trois seeds convergent exactement au même endroit. C'est pour ça que nous l'avons retenue. Et ce n'est pas un choix à la main : le programme la sélectionne automatiquement, sur le critère moyenne, puis écart-type, puis meilleure valeur. J'insiste sur un point contre-intuitif : l'exploratoire fait huit cents générations avec deux cent quarante individus, donc **beaucoup plus de calcul**, et elle n'est pas meilleure. Plus de calcul ne garantit pas un meilleur résultat. Et soyons honnêtes : trois seeds, c'est peu pour un vrai jugement statistique. Nous le disons dans nos limites.

Transition. « Regardons maintenant comment se comporte cette configuration au fil des générations. »

Question probable : « Pourquoi les configurations rapide et exploratoire ont-elles exactement les mêmes chiffres ? » **Réponse :** « Parce qu'elles tombent sur les deux mêmes attracteurs — trois mille cent cinquante-sept et trois mille trois cent trente-quatre — simplement sur des seeds différents. La rapide échoue au seed onze, l'exploratoire au seed vingt-deux. Le résultat, c'est que les trois valeurs forment le même ensemble, donc toutes les statistiques coïncident. Ce n'est pas un bug : c'est le signe que notre instance a deux optima locaux très marqués. »

SLIDE 14 — Convergence

Durée : 1 min 05

Cette courbe montre la meilleure distance connue au fil des générations. Elle part très haut — autour de six mille quatre cents kilomètres — parce que la population initiale est complètement aléatoire. Et elle chute très vite. L'essentiel du progrès se joue dans les **soixante-dix premières générations** : la meilleure tournée est atteinte dès la génération **soixante-sept**. Ensuite, plus rien pendant quatre cent cinquante générations. La ligne bleue en pointillés, c'est le résultat de Christofides. On voit que le génétique le dépasse très tôt. Et voici le point le plus important de cette slide : une courbe plate ne prouve **pas du tout** que nous avons trouvé l'optimum global. Elle prouve seulement que l'algorithme **ne progresse plus**. Il peut être coincé dans un **optimum local** — la meilleure solution de son voisinage immédiat, sans être la meilleure possible. C'est exactement ce qui arrive aux seeds qui s'arrêtent à trois mille trois cent trente-quatre kilomètres : leur courbe est tout aussi plate, et pourtant ils sont cent soixante-dix-sept kilomètres au-dessus.

Transition. « Voici la tournée que cette configuration propose à Théobald. »

Question probable : « Alors pourquoi faire cinq cent vingt générations ? » **Réponse :** « Bonne question, et c'est une de nos améliorations identifiées : les quatre cent cinquante dernières ne servent à rien. Un critère d'arrêt du type "stopper après cent générations sans amélioration" diviserait le temps de calcul par trois sans rien perdre. »

SLIDE 15 — Résultat génétique

Durée : 50 s

Voici la tournée du génétique. **Trois mille cent cinquante-sept virgule treize kilomètres.** Soit **deux cent quatre-vingt-huit kilomètres et demi de moins** que Christofides, c'est-à-dire **huit virgule trente-sept pour cent** d'amélioration. Si vous comparez les deux cartes, la différence se voit surtout dans le sud-est. Christofides fait un aller-retour vers Nice avant de redescendre sur Marseille : deux arêtes se croisent. Le génétique, lui, enchaîne proprement Nîmes, Marseille, Toulon, Nice, puis remonte par Grenoble. **C'est là que se jouent les deux cent quatre-vingt-huit kilomètres.** Et une formulation à laquelle je tiens : trois mille cent cinquante-sept kilomètres, c'est la **meilleure tournée que nous ayons observée**. Pas l'optimum. Nous n'avons pas calculé l'optimum exact, donc nous ne pouvons pas l'affirmer.

Transition — PASSAGE DE RELAIS.

« Yannis va maintenant mettre les deux méthodes face à face, sur tous les critères. »

Question probable : « Donc le génétique est meilleur ? » **Réponse :** « Sur cette instance et avec nos réglages, oui, de huit virgule trente-sept pour cent. Mais pas en général : nos seeds défavorables donnent trois mille trois cent trente-quatre kilomètres, c'est-à-dire **moins bien** que Christofides. Et il est environ mille fois plus lent. »

YANNIS — Slide 16

SLIDE 16 — Comparaison finale

Durée : 1 min 20

Merci Lisa. Voici le tableau de décision, sur les quatre critères demandés par le sujet. **Distance :** le génétique gagne, deux cent quatre-vingt-huit kilomètres de moins. **Temps :** Christofides écrase le génétique. Une milliseconde et demie contre un peu plus d'une seconde par exécution — un facteur de l'ordre de **mille**. **Déterminisme :** Christofides donne toujours exactement le même résultat. Le génétique dépend du seed. **Garantie :** Christofides ne dépasse jamais une fois et demie l'optimum. Le génétique n'offre aucune borne. **Paramétrage :** Christofides n'a presque rien à régler, le génétique a six hyperparamètres. Sur la **facilité d'implémentation**, c'est nuancé, et je veux le dire honnêtement. Christofides demande six briques distinctes, c'est plus long à écrire et il faut plus de bagage théorique — mais une fois écrit, il n'y a plus rien à faire. Le génétique s'écrit vite, mais le vrai travail commence après : il faut le régler, le benchmarker, choisir une configuration. Au total, le temps passé a été comparable. Et le point que je veux souligner : le fait que le génétique gagne ici ne veut **pas** dire qu'il gagne toujours. La garantie de Christofides est un **plafond**, pas une promesse d'être le meilleur. Le génétique, lui, n'a aucun plafond.

Transition — PASSAGE DE RELAIS.

« Romain va maintenant présenter les limites de notre étude. »

Question probable : « Pourquoi le génétique peut-il battre Christofides s'il n'a aucune garantie ? » **Réponse :** « Parce que la garantie de Christofides dit "je ne ferai jamais pire que un virgule cinq fois l'optimum", pas "je ferai mieux que tout le monde". Christofides suit une construction imposée — un

arbre, un couplage, un ordre eulérien — et cette construction est rigide : elle ne peut pas revenir en arrière pour décroiser deux arêtes. Le génétique explore librement et peut tomber sur un arrangement que la construction ne pouvait pas produire. »

ROMAIN — Slides 17 et 18

SLIDE 17 — Limites et améliorations

Durée : 1 min 20

*Je reprends la main, parce qu'un projet honnête doit savoir dire ce qu'il **ne prouve pas**. Première limite, la plus concrète : nos distances sont **à vol d'oiseau**. Théobald ne volera pas. Sa tournée réelle serait quinze à trente pour cent plus longue. Deuxième : nous n'avons **jamais calculé l'optimum exact**. Nous pouvons l'encadrer, mais pas le nommer. Troisième : **trois seeds**, c'est assez pour départager nos trois configurations, ce n'est pas assez pour une conclusion statistique. Quatrième : nos hyperparamètres sont **empiriques**, issus du benchmark. Aucune théorie ne dit que cent soixante individus et vingt-deux pour cent de mutation sont les bons réglages. Et cinquièmement, tout cela vaut pour **une instance de vingt villes**, pas pour le TSP en général. À droite, ce que nous ferions ensuite. Le plus rentable serait d'ajouter une recherche locale **deux-opt** après le génétique : elle décroise les arêtes et améliore presque toujours le résultat, pour un coût très faible. On pourrait aussi **injecter la tournée de Christofides** dans la population initiale du génétique, ce qui garantirait de partir déjà sous trois mille quatre cent quarante-cinq kilomètres. Et évidemment, passer à trente seeds.*

Transition. « J'enchaîne sur la façon dont nous nous sommes organisés tous les trois. »

Question probable : « Quelle est votre plus grosse faiblesse ? » **Réponse :** « Le benchmark à trois seeds. Il nous suffit pour départager nos configurations de façon argumentée, mais il ne permet aucune affirmation statistique. Et juste derrière : nos trois configurations font varier plusieurs paramètres à la fois, donc on ne peut pas isoler l'effet de chacun. »

SLIDE 18 — Organisation

Durée : 55 s

*Le sujet demande que notre organisation apparaisse dans la présentation. La voici. Nous avons travaillé sur un tableau **Trello** à trois colonnes, avec une règle simple : **une carte égale un livrable vérifiable**, pas une intention. Dans la colonne « terminé », vous retrouvez toute la chaîne technique : le CSV des vingt villes, Haversine, le graphe complet, Prim, Christofides, le génétique, le benchmark, les cartes et l'analyse comparative. Côté répartition : je me suis occupé des données, de Haversine, du graphe et du dépôt. Yannis a pris toute la partie Christofides. Lisa a fait l'algorithme génétique, le benchmark et les visualisations. Les tests unitaires, l'analyse comparative et les slides, nous les avons faits ensemble. Tout est public sur le dépôt GitHub **travelling-merchant**, avec un README qui reprend le contexte, les deux algorithmes et la conclusion.*

Transition — PASSAGE DE RELAIS.

« Lisa va conclure avec notre recommandation pour Théobald. »

Question probable : « Comment avez-vous réparti concrètement le travail ? » **Réponse :** « Par blocs indépendants, pour pouvoir avancer en parallèle : les données et la modélisation d'un côté, les deux algorithmes de l'autre. On se retrouvait à chaque séance pour revoir le tableau et intégrer. Les tests unitaires ont beaucoup aidé : ils garantissaient que l'intégration ne cassait rien. »

LISA — Slide 19

SLIDE 19 — Conclusion

Durée : 1 min 25

Notre recommandation pour Théobald tient en deux temps. **Pour tracer sa route, nous recommandons l'algorithme génétique** : trois mille cent cinquante-sept kilomètres, avec une configuration qui s'est montrée stable sur nos trois seeds. C'est concrètement deux cent quatre-vingt-huit kilomètres économisés à chaque tournée. Mais nous gardons **Christofides comme filet de sécurité**. Il donne une très bonne tournée en une milliseconde et demie, sans aucun réglage et sans hasard, et il sert de référence pour valider le génétique. Si Théobald devait recalculer sa tournée en permanence, ou avec beaucoup plus de villes, c'est Christofides que nous choisirions. Et pour finir, le schéma du haut montre ce que nous savons **vraiment**. L'arbre couvrant minimal — deux mille six cent soixante-cinq kilomètres — est une borne inférieure **prouvée** de l'optimum. Notre meilleure tournée — trois mille cent cinquante-sept — est une borne supérieure, puisque c'est une tournée valide. L'optimum est donc quelque part entre les deux, dans une fenêtre de quatre cent quatre-vingt-douze kilomètres. (ralentir) Nous ne livrons pas la tournée **optimale**. Nous livrons la meilleure tournée que nous **sachions construire** — et nous savons dire exactement ce qui nous en sépare. Merci de votre attention. Nous sommes prêts pour vos questions.

Question probable : « Si vous deviez ne garder qu'une méthode ? » **Réponse :** « Christofides. Parce qu'une garantie qui tient toujours vaut mieux qu'un gain de huit pour cent qui tient parfois. Mais la vraie réponse, c'est de ne pas choisir : on lance Christofides pour avoir une borne garantie en une milliseconde, et on l'injecte comme graine du génétique. On obtient alors le meilleur des deux. »

Annexe — Répétition et secours

Chronométrage cible

Bloc	Durée	Cumul
Romain — slides 1 à 5	4:40	4:40
Yannis — slides 6 à 10	6:05	10:45
Lisa — slides 11 à 15	5:55	16:40
Yannis — slide 16	1:20	18:00

Bloc	Durée	Cumul
Romain — slides 17 et 18	2:15	20:15
Lisa — slide 19	1:25	21:40

Version 18 minutes — coupez ceci

Coupe	Gain
Slide 5 : la supprimer, son contenu est repris slide 16	−45 s
Slide 14 : garder « le meilleur est atteint dès la génération 67, et une courbe plate ne prouve pas l'optimum »	−40 s
Slide 3 : ne développer que « complet » et « 190 arêtes »	−25 s
Slide 12 : ne détailler que OX et l'inversion	−35 s
Slide 18 : ramener à 30 s	−25 s
Slide 17 : 3 limites et 2 améliorations au lieu de 5 et 5	−35 s
Total récupéré	≈ 3 min 25

Version 15 minutes — coupez en plus

Coupe	Gain
Slide 2 : aller droit au 19! sans l'exemple des 4 ans	−25 s
Slide 4 : garder Haversine + l'inégalité triangulaire, couper les alternatives	−30 s
Slide 7 : couper la comparaison Prim/Kruskal/Dijkstra (la garder en réserve pour les questions)	−30 s
Slide 8 : énoncer le lemme des poignées de main sans le justifier	−25 s
Slide 11 : couper l'explication de l'ancrage de Paris (à garder pour les questions)	−25 s
Slide 13 : ne pas détailler les deux attracteurs	−25 s
Total supplémentaire	≈ 2 min 40

⚠ **Ne coupez jamais les slides 9, 16 et 19.** Ce sont celles qui portent la démonstration (le facteur 1,5), la comparaison demandée par le sujet, et la recommandation. Ne coupez pas non plus la slide 18 entièrement : le Trello est **exigé** par le sujet.

Si la démo est demandée

```
python main.py
```

Puis ouvrir dans l'ordre :

1. `results/route_christofides.html`
2. `results/route_genetique.html`
3. `results/summary.json`

Prévenez : « L'exécution prend une trentaine de secondes, surtout à cause du benchmark : neuf exécutions du génétique. »

Les cinq phrases de secours

Si vous perdez le fil, l'une de ces cinq phrases relance toujours :

1. « Ce qu'il faut retenir de cette étape, c'est qu'elle corrige le défaut de la précédente. »
2. « Toutes ces valeurs viennent de l'exécution réelle du programme, elles sont dans `results/summary.json`. »
3. « Ce que nous pouvons affirmer, c'est que l'optimum est entre 2 665 et 3 157 kilomètres. »
4. « Sur cette instance et avec nos réglages — je précise, parce que ça ne se généralise pas. »
5. « Je ne suis pas certain de ce point, mais voici comment je raisonnerais... »

Rappel des trois formulations obligatoires

1. « **observée** » et jamais « optimale ».
2. « **au plus 1,5 × l'optimum** » et jamais « 1,5 fois l'optimum ».
3. « **sur cette instance** » chaque fois que vous comparez les deux méthodes.